CE/CZ4123/SC4023 Big Data Management

Semester Group Project Report

HDB Resale Flat Column-Store Query Engine

Pham Ba Cong (Matriculation No.: U2331760J)
Tobias Glatz (Matriculation No.: N2504962B)
Aylin Stecher (Matriculation No.: N2500024C)

1 Data Storage

1.1 Column-Store Architecture

Our program implements a column-oriented storage engine from scratch in Python, without relying on external libraries such as pandas or SQL databases. In contrast to row-oriented storage where each record is stored as a contiguous unit, our `ColumnStore` class stores each attribute as an independent array. This layout provides two key advantages for analytical queries: (1) only the columns relevant to a query are accessed, avoiding unnecessary I/O on irrelevant fields, and (2) iterating over a single column array is cache-friendly since values are contiguous in memory.

Figure 1 illustrates the difference. When computing `MIN(price_per_sqm)` with a filter on `town`, the column store only touches the `town` and `price_per_sqm` arrays, while a row store would load all 10+ fields per row.

Figure 1: Row-oriented vs. column-oriented layout. Column store (right) stores each attribute as an independent array.

1.2 Dictionary Encoding for String Columns

String columns such as `town`, `flat_type`, and `flat_model` contain many repeated values. Our `Dictionary` class implements *dictionary encoding*: each unique string is assigned a sequential integer code, and the column array stores only these codes. This yields two benefits: (a) filtering by `town == "BEDOK"` becomes an integer comparison, which is significantly faster; (b) memory usage is reduced since each row stores a 4-byte integer rather than a variable-length string. The dataset contains 26 distinct towns and 7 flat types, making dictionary encoding highly effective.

1.3 Schema, Derived Columns, and File I/O

We define an explicit schema (column name → `int`/`float`/`str`) to avoid per-row type inference. After loading, three derived columns are added: `year`, `month_num` (both parsed from `month`), and `price_per_sqm` (`resale_price / floor_area_sqm`). The CSV is loaded via Python's `csv` module using direct per-column appends (reducing load time from 4.24 s to 2.73 s, 1.55× faster). String values are dictionary-encoded on the fly.

1.4 Exception Handling

The program skips malformed CSV rows, returns -1 for unknown dictionary values (causing an empty result), and reports "No result" when no valid (x, y) pairs are found.

2 Data Processing

2.1 Query Condition Derivation

Query conditions are derived from the matriculation number via the `parse_matric()` function:

- **Target year**: last digit d; if $d \geq 5$ then $2010 + d$, else $2020 + d$.
- **Start month**: second-to-last digit; 0 maps to October (10).
- **Towns**: all unique digits mapped through a lookup table (Table 1).

For `U2331760J`: target year = 2020, start month = June (06), matched towns = BEDOK, BUKIT PANJANG, CLEMENTI, CHOA CHU KANG, PASIR RIS, TAMPINES.

Digit	0	1	2	3	4	5	6	7	8	9
Town	BDK	BKP	CLM	CCK	HGN	JRW	PSR	TPN	WDL	YSH

Table 1: Digit-to-town mapping (abbreviated). BDK=BEDOK, BKP=BUKIT PANJANG, CLM=CLEMENTI, CCK=CHOA CHU KANG, HGN=HOUGANG, JRW=JURONG WEST, PSR=PASIR RIS, TPN=TAMPINES, WDL=WOODLANDS, YSH=YISHUN.

2.2 Column-at-a-Time Filtering

Our `Query` class applies filter predicates one column at a time via a chainable API:

```
.filter("year","==",y).filter("town","in",towns)...min("price_per_sqm")
```

Only the relevant column arrays are iterated; surviving row indices are progressively narrowed. For string columns, values are resolved to dictionary codes before comparison, making filters integer set-membership tests.

2.3 Naive Baseline (`run_query_naive`)

The naive implementation issues an independent query for every (x, y) pair ($8 \times 71 = 568$ queries). Each query re-scans the full dataset through the `Query` API. While correct and simple, this approach performs significant redundant work since queries for adjacent (x, y) pairs overlap heavily.

2.4 Optimized Query (`run_query`)

The optimized implementation eliminates redundancy through two techniques:

Optimization 1: Incremental Month Accumulation. Instead of re-filtering from scratch for each x value, we pre-filter all rows matching the target year and matched towns, then group them by month. As x increases from 1 to 8, we only add the new month's rows to an accumulator:

- $x = 1$: candidates from month `start_month`
- $x = 2$: reuse $x = 1$ candidates + add month `start_month + 1`
- $x = k$: reuse $x = k-1$ candidates + add month `start_month + k - 1`

We maintain a `best_price_at_area[a]` array that tracks the minimum `price_per_sqm` seen for each exact floor area value. When a new month is added, only the new month's rows update this array.

Optimization 2: Area Sweep with Running Minimum. For a given x, the query asks: for each y, what is min(price_per_sqm) where area $\geq y$? Since the condition area $\geq y$ includes all areas from y up to the maximum, we perform a single backward sweep from `max_area` down to `y_min`, maintaining a running minimum. This answers all 71 values of y in $O(\text{max_area})$ time, rather than the $O(n \times 71)$ time of the naive approach.

2.5 Vectorized Out-of-Core Loading

For out-of-core scenarios, `iter_load_vectors` reads the CSV in fixed-size batches (4,096 rows), building a separate `ColumnStore` per batch. Each batch runs the same transform/layout/zone-map pipeline and `run_query` as the full-load path. Partial (x, y) minima are merged across batches, keeping the working set bounded to one batch plus merge state. Table 2.8 compares total time and peak memory.

2.6 Late Materialization

Throughout the pipeline, rows are represented as indices into column arrays. Full row reconstruction (including string decoding) is deferred until the final output stage via `materialize_row()`. This avoids the cost of decoding dictionary-encoded strings for the thousands of rows that are ultimately filtered out.

2.7 Storage and Query Optimizations

We implemented two additional column-store optimizations to improve query performance.

Sorted Column Layout. All columns are physically sorted by `(year, month_num, town)` via `sort_by()`. This clusters records by the most common filter predicates. When combined with zone maps (below), sorted layout enables the zone map to skip entire contiguous regions of irrelevant data.

Zone Maps (Min-Max Indexes). For each block of 4,096 rows, we store the minimum and maximum value of key columns (`year`, `month_num`, `town`, `floor_area_sqm`). Before scanning any block, the query engine checks whether the block's min-max range overlaps the filter predicate. Non-overlapping blocks are skipped entirely. Zone maps produced the largest single-optimization speedup: the optimized query dropped from 0.769 s to 0.063 s (**12.2**× faster), and peak memory from 10.07 MiB to 1.28 MiB (**7.9**× lower).

Table 2 presents the isolated and combined impact of each optimization.

Config	Opt. query	Naive query	Opt. query peak mem
Baseline (none)	0.769 s	16.379 s	10.07 MiB
Sorted layout only	0.616 s	15.239 s	10.07 MiB
Zone maps only	**0.063 s**	1.465 s	**1.28 MiB**
Full all (sort+zonemap)	**0.063 s**	**1.454 s**	**1.28 MiB**

Table 2: Isolated and combined impact of each optimization (10-run averages). Zone maps dominate: 12.2× faster optimized queries vs. baseline, while sorting adds little once zone maps are enabled.

2.8 Performance Comparison

Tables 3 and 2.8 summarise full-load and vectorized modes (10-run averages, sorted layout + zone maps).

Mode	Load (s)	Query (s)	Total (s)	Query peak mem
Full Load + Naive Query	3.267	1.343	4.610	1.28 MiB
Full Load + Optimized Query	3.267	0.060	3.327	1.28 MiB
Vectorized + Optimized	—	—	7.321	0.18 MiB

Table 3: Execution modes. Optimized query is 22.5× faster than naive; vectorized mode trades 2.2× slower total time for 7.1× lower query peak memory.

Vectorized mode is slower because chunking repeats transform/layout/zone-map setup per batch and adds merge overhead, whereas the full-load path applies these steps once over the entire dataset.

3 Experiment Result

3.1 Execution Output

We executed the program using matriculation number `U2331760J`. The derived query parameters are:

- Target year: 2020, start month: June (06)
- Matched towns: BEDOK, BUKIT PANJANG, CLEMENTI, CHOA CHU KANG, PASIR RIS, TAMPINES
- $x \in [1, 8]$, $y \in [80, 150]$, threshold: 4725
- Input rows loaded: 259,237

The program successfully executed all three query modes. Selected output metrics:

- Load time: 3.267 s; 259,237 rows, 10 columns loaded
- Example MIN query: `CHOA CHU KANG, Block 295, 118 sqm, $325,000, price/sqm = 2754.24`
- Records in 2020 for matched towns: 6,532
- Average resale price in BEDOK (2020): $411,303
- Mode 1 (optimized): **568** valid (x, y) pairs, query time 0.060 s, peak 1.28 MiB
- Mode 2 (naive): **568** valid (x, y) pairs, query time 1.343 s; speedup 22.5×; results `MATCH`
- Mode 3 (vectorized): **568** valid (x, y) pairs, 64 chunks, query time (sum of chunks) 0.050 s, total 7.321 s, query peak 0.18 MiB; results `MATCH`

All three modes consistently report 568 valid (x, y) pairs across all benchmark runs, and the naive/optimized verification result is `MATCH` in every run.

```
❯ python3 source/main.py
=== HDB Resale Flat Column-Store Query Engine ===
Matriculation: U2331760J
Target year:   2020
Start month:   06
Matched towns: BEDOK, BUKIT PANJANG, CLEMENTI, CHOA CHU KANG, PASIR RIS, TAMPINES
x range:       1 to 8
y range:       80 to 150
Threshold:     4725

Loading data...
Loaded 259237 rows, 10 columns from /mnt/e/Desktop/SC4023/project/ResalePricesSingapore.csv
  month (str): 132 distinct values
  town (str): 26 distinct values
  flat_type (str): 7 distinct values
  block (str): 2747 distinct values
  street_name (str): 577 distinct values
  storey_range (str): 17 distinct values
  flat_model (str): 21 distinct values
  Load time: 3.267s
  Sorted layout enabled: True
  Zone maps enabled: True

Example query: MIN(price_per_sqm) WHERE year=2020, month in [6,8], town in matched, area >= 85
  Result: CHOA CHU KANG, Block 295, 118 sqm, $325000, price/sqm = 2754.24

  Records in 2020 for matched towns: 6532
  Average resale price in BEDOK (2020): $411,303
```

(a) Configuration, loading, and example query result.

```
==================================================
MODE 1: Full Load + Optimized Query
==================================================
Running optimized query...
  Pre-filtered to 6532 candidate rows
  Found 568 valid (x, y) pairs
  Optimized query time: 0.060s
  Optimized query peak memory: 1.28 MiB

Writing results...
Results written to /mnt/e/Desktop/SC4023/project/ScanResult_U2331760J.csv
  Total valid (x, y) pairs: 568
  Write time: 0.010s

==================================================
MODE 2: Naive Baseline (Generic Query API)
==================================================
  Naive: found 568 valid (x, y) pairs
  Naive query time: 1.343s
  Verification: naive and optimized results MATCH

  Speedup: 22.5x faster with optimized

==================================================
MODE 3: Vectorized (Fair Chunked Full Pipeline)
==================================================
  Chunks processed: 64
  Rows scanned: 259237
  Vectorized total time (load+transform+layout+query): 7.321s
  Vectorized query time (sum of chunks): 0.050s
  Vectorized query peak memory (max over chunks): 0.18 MiB
Results written to /mnt/e/Desktop/SC4023/project/ScanResult_U2331760J_vectorized.csv
  Total valid (x, y) pairs: 568
  Verification: full-load and fair vectorized results MATCH

==================================================
TIMING SUMMARY
==================================================
  Full load:           3.267s
  Naive query:         1.343s
  Optimized query:     0.060s
  Vectorized load+qry: 7.321s
  Output write:        0.010s

  Normal query peak memory:     1.28 MiB
  Vectorized query peak memory: 0.18 MiB

Done!
```

(b) Mode 1/2/3 results and timing summary.

Figure 2: Terminal output of `python3 source/main.py`. All modes report 568 valid (x, y) pairs.

3.2 Output Validation with Excel

To validate correctness, we selected several (x, y) pairs and independently verified the results using Microsoft Excel. We applied the equivalent filter conditions and computed `MIN(Resale_Price / Floor_Area)` to confirm the minimum price per square meter.

Validation: We added helper columns (`year`, `month_num`, `price_per_sqm`, `town_match`) and used a `MINIFS` formula over an Excel Table to replicate each query's filters. Results matched the program output exactly (Table 4).

(x, y)	Program Output	Excel Result	Match
(1, 80)	2897	2897	✓
(3, 100)	2754	2754	✓
(5, 120)	2897	2897	✓
(7, 85)	2336	2336	✓

Table 4: Comparison of program output vs. Excel-computed results for selected (x, y) pairs.

3.3 Output File

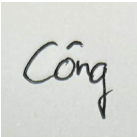
The generated `ScanResult_U2331760J.csv` file contains all 568 valid (x, y) pairs sorted in increasing order of x, then y, as required. Each row includes the full record details for the transaction with the minimum price per square meter for that (x, y) pair. A vectorized variant `ScanResult_U2331760J_vectorized.csv` is also produced, which produces identical results using the out-of-core loading path.

CONTRIBUTION FORM

Group Number: 35

Name	**Matriculation Number**	**Detailed Individual Contribution**	**Percentage (100% in total)**
Pham Ba Cong	U2331760J	Discussion, implementation, report writing	34%
Tobias Glatz	N2504962B	Discussion, testing code, report writing	33%
Aylin Stecher	N2500024C	Discussion, testing code, report writing	33%

Name and Signature from all group members:



Pham Ba Cong (Member 1)

Tobias Glatz (Member 2)

Aylin Stecher (Member 3)